

Version 1 Review – Sustainable Lifestyle Calculator

1. Project Overview

The Sustainable Lifestyle Calculator is a Python-based command-line application designed to help users evaluate and improve their sustainability habits. The application collects user lifestyle data across multiple sustainability-related categories such as transportation, electricity usage, water consumption, recycling habits, and plastic usage.

Using this data, the application calculates an overall eco score and provides personalized feedback to help users understand the environmental impact of their lifestyle choices. The goal of the project is to promote awareness and encourage users to adopt more sustainable daily habits through measurable insights and actionable recommendations.

The application is intended for general users who want a simple and accessible way to assess their sustainability practices without requiring advanced technical knowledge.

2. Current Implementation Status

At this stage, the core functionality of the application has been successfully implemented, and the application is fully functional in command-line form.

Features implemented so far

- Command-line interface (CLI) for user interaction
- Collection of sustainability data across multiple categories:
 - Transportation (car, motorcycle, bus, bicycle, walking)
 - Electricity usage
 - Water consumption
 - Recycling habits
 - Single-use plastic usage
- Input validation for numerical entries

- Category-wise sustainability score calculation
- Weighted overall eco score generation
- Sustainability achievement classification system
- Personalized category-based feedback generation
- CSV-based score storage for historical tracking
- Progress visualization using matplotlib graphs

Features partially implemented / pending

- More advanced sustainability analytics across multiple sessions
- More detailed recommendation logic based on long-term trends
- Optional graphical user interface (GUI)
- Expansion into additional sustainability categories such as food habits or waste segregation

The current application can successfully collect user data, compute sustainability scores, store results, and visualize progress over time.

3. Demonstration of Current Application

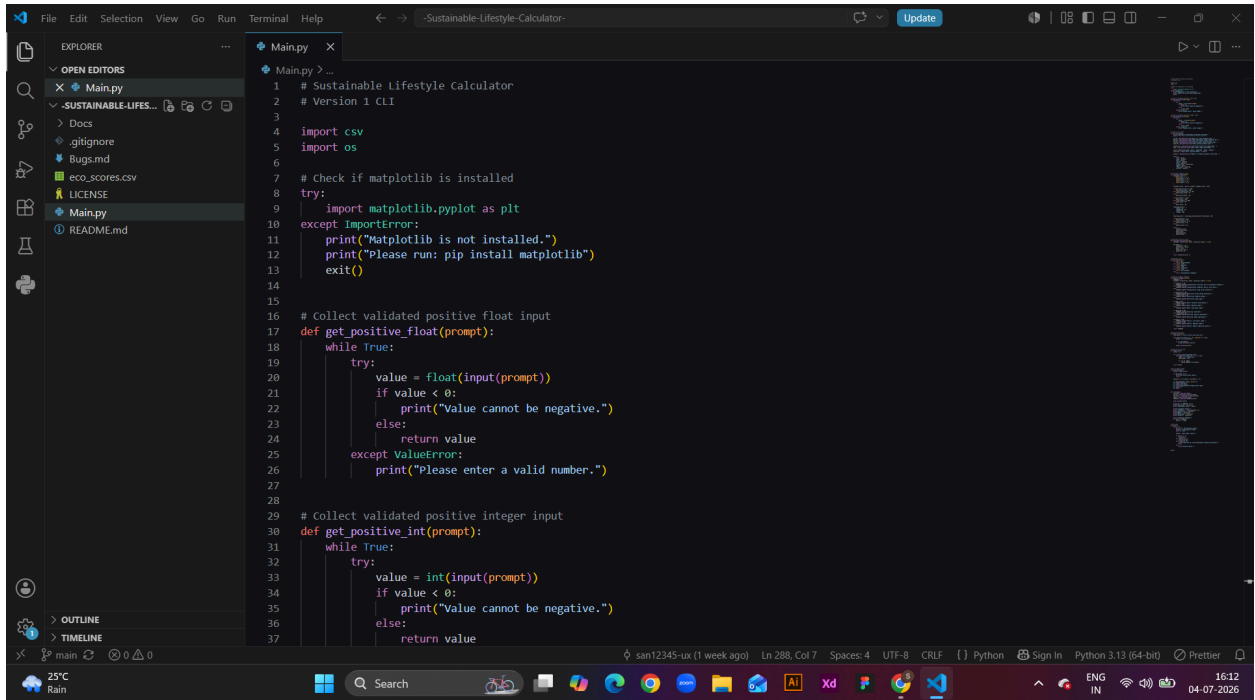
The application currently runs through the terminal using Python.

Typical workflow

1. User launches the application
2. Main menu is displayed with options to calculate score or view progress graph
3. User enters sustainability-related inputs
4. Application calculates category scores
5. Overall eco score is generated
6. Sustainability achievement level is assigned
7. Personalized feedback is displayed
8. Eco score is saved to CSV
9. User can visualize score history using a progress graph

Screenshots

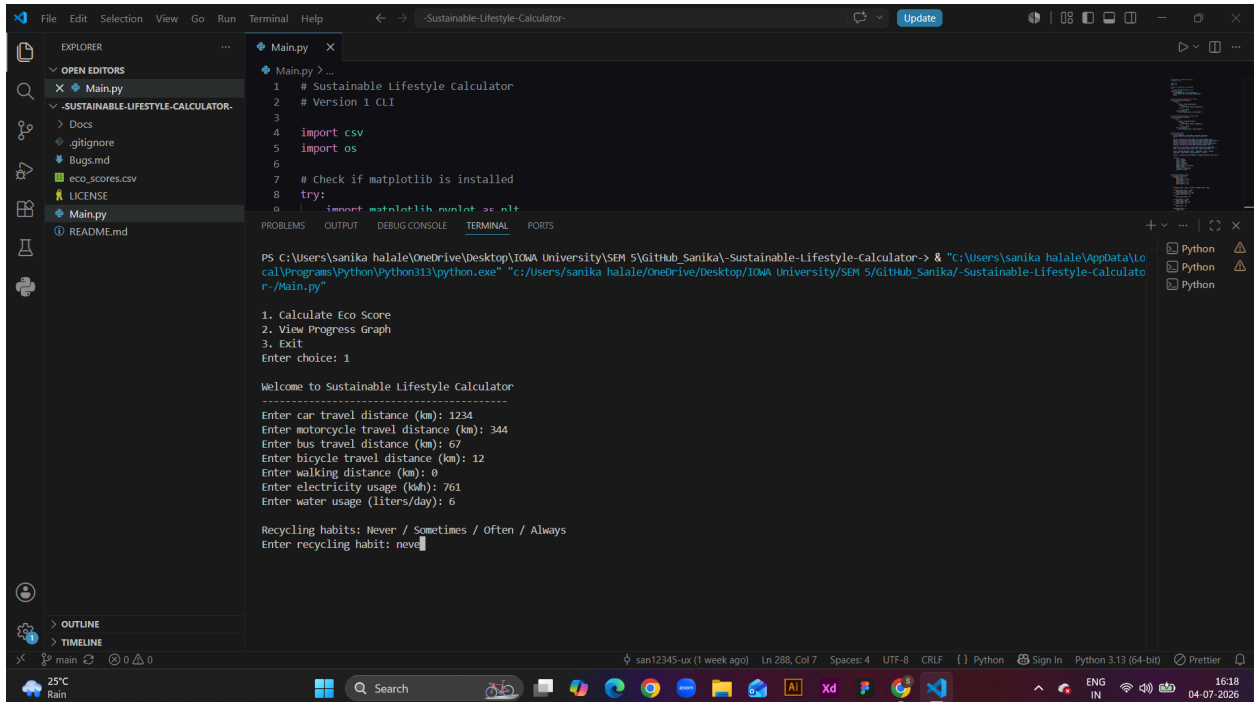
Screenshot 1: CLI code



This screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure. The main editor window shows the code for `Main.py`. The code includes imports for `csv` and `os`, a check for `matplotlib` installation, and two functions: `get_positive_float` and `get_positive_int`. The `get_positive_float` function prompts the user for a positive float value, and the `get_positive_int` function prompts for a positive integer value. The code is written in Python and uses a dark theme.

```
1 # Sustainable Lifestyle Calculator
2 # Version 1 CLI
3
4 import csv
5 import os
6
7 # Check if matplotlib is installed
8 try:
9     import matplotlib.pyplot as plt
10 except ImportError:
11     print("Matplotlib is not installed.")
12     print("Please run: pip install matplotlib")
13     exit()
14
15
16 # Collect validated positive float input
17 def get_positive_float(prompt):
18     while True:
19         try:
20             value = float(input(prompt))
21             if value < 0:
22                 print("Value cannot be negative.")
23             else:
24                 return value
25         except ValueError:
26             print("Please enter a valid number.")
27
28
29 # Collect validated positive integer input
30 def get_positive_int(prompt):
31     while True:
32         try:
33             value = int(input(prompt))
34             if value < 0:
35                 print("Value cannot be negative.")
36             else:
37                 return value
```

Screenshot 2: User entering sustainability data



This screenshot shows the Visual Studio Code editor with the file explorer on the left displaying the project structure. The main editor window shows the code for `Main.py`. The code includes imports for `csv` and `os`, a check for `matplotlib` installation, and two functions: `get_positive_float` and `get_positive_int`. The code is written in Python and uses a dark theme. The terminal window at the bottom shows the output of the program, including the welcome message and the prompts for user input.

```
PS C:\Users\sanika halale\OneDrive\Desktop\IOWA University\SEM 5\Github_Sanika\-Sustainable-Lifestyle-Calculator-> & "C:\Users\sanika halale\AppData\Local\Programs\Python\Python313\python.exe" "c:\Users\sanika halale\OneDrive\Desktop\IOWA University\SEM 5\Github_Sanika\-Sustainable-Lifestyle-Calculator-/Main.py"

1. Calculate Eco Score
2. View Progress Graph
3. Exit
Enter choice: 1

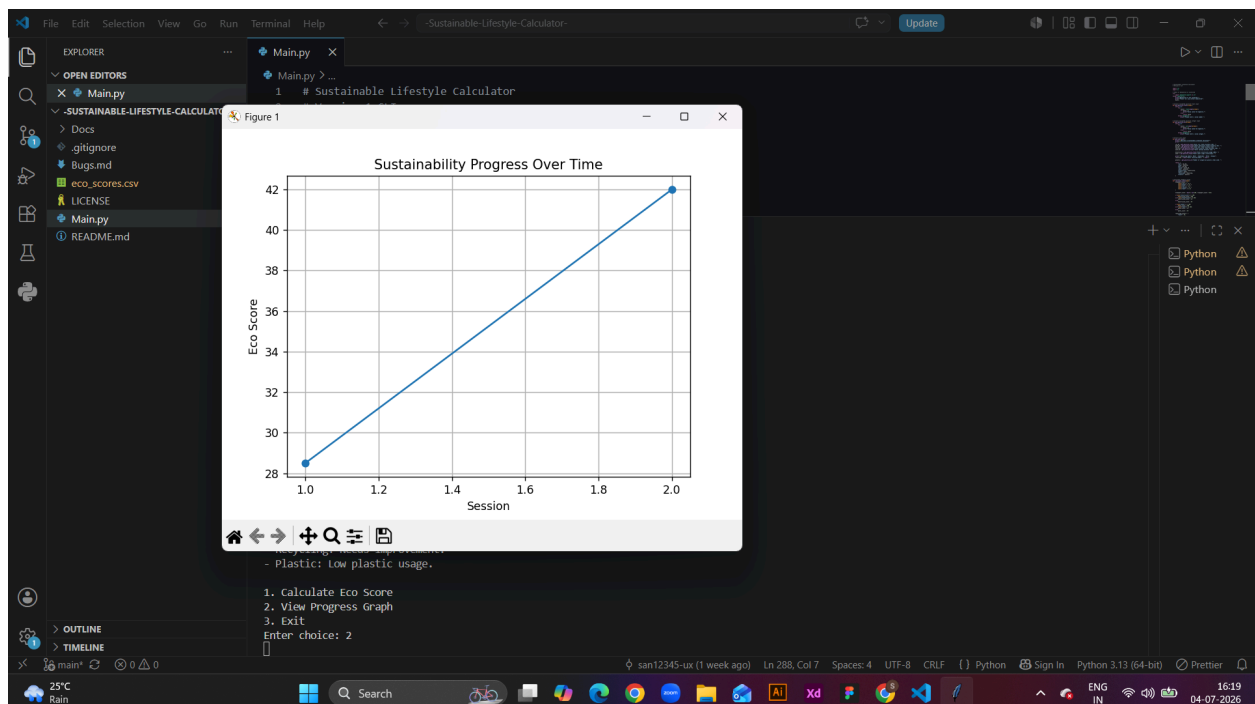
Welcome to Sustainable Lifestyle Calculator
-----
Enter car travel distance (km): 1234
Enter motorcycle travel distance (km): 344
Enter bus travel distance (km): 67
Enter bicycle travel distance (km): 12
Enter walking distance (km): 0
Enter electricity usage (kwh): 761
Enter water usage (liters/day): 6

Recycling habits: Never / Sometimes / Often / Always
Enter recycling habit: neve
```

Screenshot 3: Eco score and feedback results

```
1 # Sustainable Lifestyle Calculator
2 # Version 1 CLI
3
4 import csv
5 import os
6
7 # Check if matplotlib is installed
8 try:
9     import matplotlib.pyplot as plt
10 except ImportError:
11     print("Matplotlib is not installed. Please install it using pip install matplotlib.")
12
13 # Prompt user for input
14 print("Enter water usage (liters/day): ")
15 water_usage = input()
16
17 print("Recycling habits: Never / Sometimes / Often / Always")
18 print("Enter recycling habit: ")
19 recycling_habit = input()
20
21 print("Number of single-use plastic items used: ")
22 plastic_items = input()
23
24 # Calculate Eco Score
25 eco_score = calculate_eco_score(water_usage, recycling_habit, plastic_items)
26
27 # Display Results
28 print("----- RESULTS -----")
29 print("Eco Score: ", eco_score)
30 print("Achievement Level: Improving")
31
32 # Category Scores
33 print("Category Scores:")
34 print("Transport: 0")
35 print("Electricity: 40")
36 print("Water: 100")
37 print("Recycling: 20")
38 print("Plastic: 100")
39
40 # Detailed Feedback
41 print("Detailed Feedback:")
42 print("- Transportation: High carbon footprint.")
43 print("- Electricity: High usage.")
44 print("- Water: Excellent conservation.")
45 print("- Recycling: Needs improvement.")
46 print("- Plastic: Low plastic usage.")
47
48 # Menu
49 print("1. Calculate Eco Score")
50 print("2. View Progress Graph")
51 print("3. Exit")
52 print("Enter choice: ")
```

Screenshot 4: Progress graph generated using matplotlib



4. Challenges and Issues Encountered

1. Designing a meaningful scoring system

One of the biggest challenges was designing a scoring system that fairly represents multiple sustainability factors. Transportation, electricity usage, water consumption, and waste-related habits contribute differently to environmental impact, making it difficult to assign balanced scoring weights.

Solution

The scoring model was refined using weighted category contributions so that more environmentally impactful factors contribute proportionally to the final eco score.

2. Balancing scoring thresholds

Another challenge was determining realistic threshold ranges for category scoring. For example, deciding what qualifies as low, moderate, or high electricity or water consumption requires several iterations.

Solution

Threshold-based scoring logic was introduced to classify user behavior into meaningful performance ranges.

3. Data persistence and progress tracking

Initially, the application only generated scores during a single session, making it difficult to track improvement over time.

Solution

CSV-based storage was implemented to save eco scores across sessions, and matplotlib was integrated to visualize progress through graphs.

4. Creating useful feedback

Generating feedback that felt practical rather than generic was also challenging. Simple overall scores were not sufficient to explain specific improvement areas.

Solution

Category-based feedback was implemented so users receive targeted suggestions for each sustainability area.

5. Remaining Issues / Future Challenges

1. Limited analytics

The current graph only tracks overall eco score across sessions. It does not yet analyze category-level trends or detailed behavioral changes over time.

2. Recommendation depth

Current recommendations are rule-based and static. More adaptive recommendations based on historical data could improve usefulness.

3. User interface limitations

Although the CLI interface works well for development and testing, a graphical interface would significantly improve accessibility and user experience.

6. Areas Where Help Is Needed

At this stage, no major blockers remain and the project is progressing steadily.

However, feedback in the following areas would be helpful:

- Suggestions for improving scoring accuracy and weighting
 - Recommendations for more meaningful visual analytics
 - Guidance on prioritizing advanced features for the final version
-

7. Milestones for Version 2

Milestone 1 – Advanced Analytics

Enhance score tracking by introducing category-wise trend analysis and more detailed historical insights.

Milestone 2 – Improved Recommendation Engine

Expand recommendation logic to provide smarter and more context-aware sustainability suggestions based on user history.

Milestone 3 – Feature Expansion

Add additional sustainability metrics such as food consumption, waste segregation, or carbon footprint estimation.

Milestone 4 – GUI Development

Develop a graphical interface to improve usability and make the application more accessible to non-technical users.

Milestone 5 – Final Testing and Documentation

Perform comprehensive testing, fix bugs, improve code quality, and complete final documentation.

8. Self Reflection

Are you satisfied with your progress so far?

Yes, I am satisfied with the progress made so far. The core functionality of the application has been implemented successfully, and the project is already usable in its current state.

Do you think you can finish the project within the remaining weeks?

Yes. Since the fundamental architecture is complete, the remaining work is focused primarily on feature improvements and refinement rather than building the application from scratch.

Expectations vs. Reality

Some parts of development were easier than expected, particularly implementing the command-line interface and handling user input.

However, designing a meaningful sustainability scoring model was more difficult than

anticipated. Translating complex environmental impact factors into a fair computational scoring system required multiple iterations and careful balancing.

Light bulb moments

A major realization during development was that sustainability cannot be accurately represented using a single simple metric. Different lifestyle behaviors affect environmental impact in different ways, making weighted scoring essential.

This insight significantly influenced the design of the scoring and feedback systems and improved the overall usefulness of the application.